

PET FOOD FEEDER

Developed by

Chiş Bogdan-Raul

Table of Contents

1. Introduction
2. System Overview
3. Hardware Components
4. Software Architecture
5. Cloud Architecture (Microsoft Azure)
6. Android Application
7. Data Flow & Communication
8. Physical Design & Enclosure
9. Setup & Configuration
10. Operational Workflow
11. Security & Reliability
12. Testing & Validation
13. Challenges & Solutions
14. Future Enhancements
15. Conclusion

1. Introduction

Project Overview

The Automated Pet Food Feeder is an IoT-enabled system designed to automate pet feeding while providing remote monitoring and control via an Android application. It integrates weight sensing, servo-controlled dispensing, facial recognition and cloud-based data logging to ensure pets are fed reliably based on consumption patterns. This system is the perfect tool for keeping up with the responsibilities of a pet owner, even when away from home for an extended period.

Key Objectives

- Automate food/water dispensing at scheduled times, if food levels are low (<50g);
- Enable manual override via Android app;
- Log feeding events, water refills and cat detection in the cloud;
- Ensure plug-and-play usability with auto-boot - no complicated setup required.

Target Users

Owners of cats and small-breed dogs seeking to manage feeding schedules remotely while monitoring the pet's activity in an easy and comfortable way.

2. System Overview

Core Functionalities

- *Automated Feeding:* Dispenses food 3x daily, only if the load cell detects < 50g of food remaining at fixed hours (10:00, 15:00, 19:00);
- *Manual Control:* The Android app can trigger the food servo (HobbyKing HK15178I) or the water pump servo (K15178I SG90);
- *Cat Recognition:* Our incorporated Trust camera has the role of detecting and identifying a cat based on its appearance;
- *Data Logging:* All the meaningful events regarding the feeding process (timestamps, actions – triggering the food or the water pump servo, a cat detection) are sent to Azure, which is

Microsoft's cloud computing platform ideal for IoT to manage and analyze data from smart devices;

- *History Tracking:* The Android app also has the ability to display logs from Azure's database.

High-Level Architecture

Raspberry Pi 5 → Sensors/Servos → Azure Cloud (Data/Logic) ↔ Android App (UI/Control)

Breakdown of the High-Level Architecture:

1. Raspberry Pi 5

- This is a small, powerful computer placed **locally**;
- It connects directly to **sensors** (to gather data) and **servos** (to control physical devices).

2. Sensors/Servos

- **Sensors** send the real-world data (like measuring the grams of remaining food) to the Raspberry Pi;
- **Servos** receive commands from the Raspberry Pi to perform physical actions (like dispensing food).

3. Azure Cloud (Data/Logic)

Name	Type	Last Viewed
 chis32	Storage account	a few seconds ago
 test	Resource group	a week ago
 Azure subscription 1	Subscription	2 weeks ago

- The Raspberry Pi sends collected sensor data **to the Azure Cloud**, where data is stored, processed and analyzed;
- This cloud component also keeps historical data.

4. Android App (UI/Control)

- The user interacts with the system via an **Android app**;
- The app displays data collected from sensors and lets the user send control commands;
- It communicates **with Azure Cloud** to get updated information and send user inputs.

3. Hardware Components

3.1 Raspberry Pi 5 (8GB RAM)

- Role: Central controller for sensors, servos, camera and cloud communication;
- Specs: Broadcom BCM2712 CPU, Wi-Fi 6, USB 3.0.

3.2 Weight Sensing Unit

- Load Cell (it can accurately measure weights up to 3 kilograms): Measures the food's weight;
- HX711 Module (small electronic module designed specifically to work with load cells): It amplifies the tiny electrical signals generated by the load cell (because raw signals are too weak to read directly) and it converts these analog signals into digital data that the Raspberry Pi can read and process;
- Threshold: Triggers a refill if the weight of the remaining food in the bowl is less than 50g.

3.3 Actuators

- Food Servo: Controls the food hatch opening;
- Water Pump Servo: Manages water flow (higher torque for reliability).

3.4 Imaging System

- Trust Camera: Captures cat images at the two feeding zones (water zone and food zone);
- haarcascade_frontalcatface dataset: YOLOv5 model for real-time cat detection.

3.5 Power & Connectivity

- 5V/3A power supply for Raspberry Pi + peripherals;
- Forced Wi-Fi 6 connection to home router (static IP).

4. Software Architecture

Raspberry Pi Stack

- OS: Raspberry Pi OS Lite (64-bit);
- Languages: Python 3.9 (primary), Bash (startup scripts);
- Key Libraries:
 - o RPi.GPIO for servo control;
 - o hx711-py for weight sensing;
 - o haarcascade_frontalcatface pretrained dataset for Cat Recognition
 - o requests for Azure API calls.

Automation Script Workflow

This describes how the system runs from boot to data upload, completely hands-free. It's an orchestrated routine that combines software, hardware and cloud services.

1. *Boot:* When the device powers on, a Python script is launched automatically via systemd, a Linux service manager;
2. *Sensing:* The load cell continuously checks the weight of the food bowl every second. The system uses this to detect whether a cat has eaten, how much was eaten and whether the bowl needs refilling;
3. *Scheduling:* Cron jobs (scheduled tasks in Linux systems) trigger feeding checks at 10 AM, 3 PM and 7 PM;
4. *Camera is used along with a haarcascade_frontalcatface pretrained dataset for YoloV5*
5. *Cloud Sync:* Once all data is collected it's bundled into a JSON file (a lightweight data format). This data is then sent to Azure, Microsoft's cloud, for: logging, remote access and long-term storage.

5. Cloud Architecture (Microsoft Azure)

5.1 Components

These are the cloud services on Azure that work together to manage our smart system. Each one plays a specific role in the data pipeline - from the device to the mobile app.

- *Azure IoT Hub (gateway between the physical device and the cloud):* Device registration & bidirectional communication - the device can send data to the cloud and receive commands from it;
- *Azure JSON Database (our system's memory):* Stores logs (timestamps, event types - feeding, drinking, motion detected and cat recognition status);
- *Azure Functions:* Processes incoming JSON data and triggers database updates;
- *App Service (the bridge between the app and the backend, a reliable layer for interaction):* Hosts direct HTTP requests - the interface that the Android app uses to communicate with the system.

5.2 Data Schema (JSON Content)

```
{
  "timestamp": "2025-05-24 18:26:56",
  "event_type": "dispense",
  "item": "food",
  "weight": 140.59
},
{
  "timestamp": "2025-05-24 18:27:04",
  "event_type": "dispense",
  "item": "water",
  "weight": 147.53
},
{
  "timestamp": "2025-05-24 18:27:07.390",
  "item": "cat_detected"
},
}
```

5.3 API Endpoints (Removed - Not Applicable)

This section outlines how data flows between our device (the Raspberry Pi) and the Android app through a direct HTTP requests hosted in the cloud.

- *POST /event*: This endpoint is used by the device. It sends event data (such as feeding, motion, cat recognition) to the server;
- *GET /history*: This endpoint is used by the Android app. It fetches a list of past events, filtered by date. This is how our app reads the diary of what the system has recorded.

6. Android Application

UI Components

- Home Screen:
 - o Manual buttons to control the food and the water servos;
 - o "Auto-Feed Now" button to force scheduled check.
- History Screen:
 - o Table view of the events (date/time, action, cat detection);
 - o Filter by date range;
 - o "Close Log" button to exit the history screen.

Technical Stack

This section describes the core technologies that power the system - from the programming language to how the device connects to the cloud and the local network.

- *Programming Language*: Java;
- *Cloud Integration*: using SAS (Shared Access Signature), a secure token that grants limited-time permissions to access Azure resources. API requests from the app include a SAS token for authentication. The system securely communicates with Azure without needing a full login each time.
- *IP Handling*: User configures Raspberry Pi's static home IP during the setup of the device.

7. Data Flow & Communication

This section explains how the data moves from the Raspberry Pi all the way into the cloud and eventually into the database used in our app. It's the full pipeline from hardware to backend.

7.1 Raspberry Pi → Azure

This describes how the device captures events and sends that information up to Azure for storage and processing.

- *Raspberry Pi captures:*
 - Sensor reading (bowl weight)
 - Events (such as feeding or cat recognition)
 - Timestamp (the date and hour)
- *Raspberry Pi sends JSON payload to Azure IoT Hub*
 - Using a secure connection with a SAS token
- *Raspberry Pi directly uploads log file*
 - This function parses the JSON, checks if it's valid
 - Stores it in Azure Blob Storage
- The blob file serves as a raw event archive

7.2 Android App ↔ Azure

- The app sends a request to Azure's direct HTTP requests, asking for logs between two dates, and gets back a list of events stored in the Azure database. To get the logs the app sends a GET request, similar to this: `GET /history?start=2025-05-01&end=2025-05-31`.

7.3 App → Raspberry Pi

- The Android app can directly control the Raspberry Pi over the local Wi-Fi network, without needing the cloud. It sends a direct HTTP command to the Pi's local API server for example, to activate a motor that dispenses food or water.

Example: `http://192.168.1.50:5000/activate_servo?type=food`

8. Physical Design & Enclosure

Materials:

- Lightweight, elastic plastic sheets (ABS) for impact resistance;
- Molded glue mounts for Raspberry Pi, servos and camera.

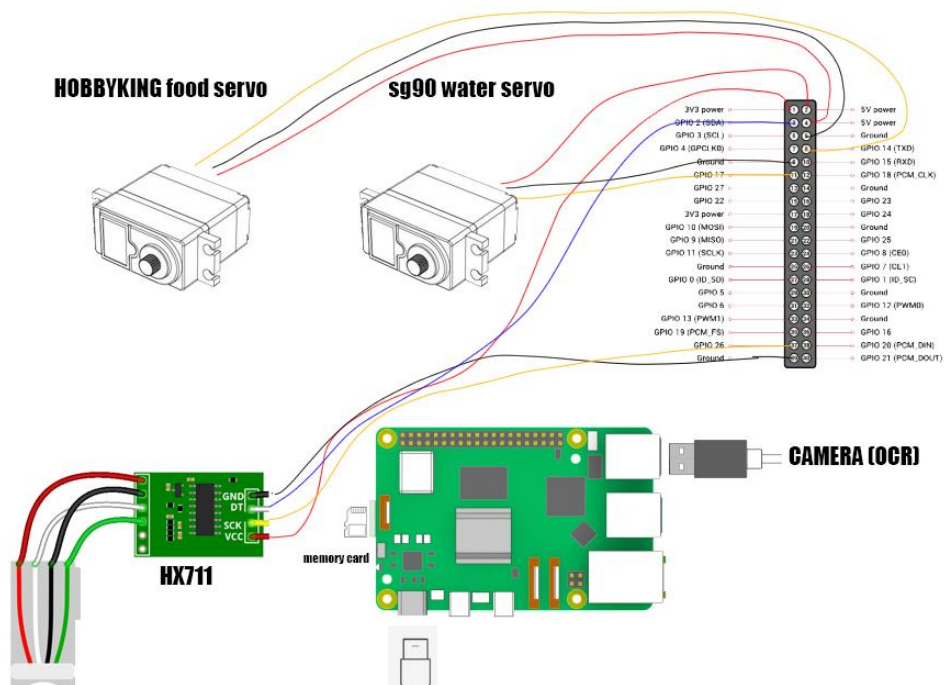
Assembly:

- Food Container: Hatch mechanism with HK15178I servo (30° rotation);
- Water Container: Sealed reservoir with SG90 servo-driven pump;
- Load Cell: Mounted under food container;
- Camera: Positioned above feeding area for facial capture.

Enclosure Features:

- Ventilated compartments for electronics;
- Waterproof seal for water pump section;
- Quick-access refill hatches.

Project Hardware Scheme:



9. Setup & Configuration

This section covers how to get the Raspberry Pi, Azure, and the Android app up and running: from Wi-Fi to cloud integration to user onboarding.

9.1 Raspberry Pi

This is how we prepared the Pi to run reliably and automatically every time it boots.

1. Wi-Fi Hardening:

- o Locking the Pi to a specific Wi-Fi network, so it will not auto-connect to something random. Therefore, a force connection to home SSID is needed:

```
network= {ssid="HomeWiFi", psk="password", priority=1}
```

2. Auto-Boot:

- o Create a systemd service to run Python script on startup. After this step the script launches every time the Pi boots.

3. Dependencies:

- o Install the Python libraries that are needed for the code:

1. RPi.GPIO: Controls GPIO pins (motors)
2. hx711: For reading the load cell (food weight)
3. opencv-python: Image processing

command: `pip install RPi.GPIO hx711 opencv-python pytesseract requests`

9.2 Azure

- Get your SAS token so the Pi can securely send data;
- Configure Function to parse JSON and write to Container.

9.3 Android App

- Initial configuration when the user installs the app: enters RPi's IP.

10. Operational Workflow

Automated Feeding Sequence

1. A time trigger (e.g., 3 PM) initiates a weight check of the remaining food;
2. If the food's weight $< 50\text{g}$:
 - o Food servo activates for 5s -> dispenses 100g of food;
 - o Camera captures image -> YOLOv5 detects cat;
 - o The feeding event is sent to Azure.
3. If the food's weight $\geq 50\text{g}$: No further action.

Manual Override

- The app sends a specific HTTP command based on our wish (to provide food or water) -> RPi activates the targeted servo immediately.

11. Security & Reliability

This is the part of our system that keeps the data safe and the system running. This section outlines how the system is protected from unauthorized access and how it recovers from basic failures.

Measures that were taken:

- Data Encryption: HTTPS for all cloud communications.
- IP Whitelisting: Azure SQL Database is locked down to only accept connections from specific IP addresses: the Raspberry Pi's home static IP, the cloud backend (e.g., Azure Function), the Android app;
- Redundancy: Retrying API calls if they fail & storing logs locally on the Pi if the cloud is temporarily unreachable;
- Wi-Fi Stability: A systemd service monitors Wi-Fi connectivity and restarts Wi-Fi on disconnect.

12. Testing & Validation

Meaningful test cases for our system:

<u>Scenario</u>	<u>Expected Result</u>
Weight < 50g at schedule	- Food dispensed;
Weight >= 50g at schedule	-No action;
Manual food trigger	-Servo activates instantly;
Camera detection	-Cat detected stored in Azure Log File
Azure offline	-Event stored locally.

13. Challenges & Solutions

The challenge we encountered: the servo motor shook and twitched unpredictably, which could have led to inaccurate movements, wasted energy, or even mechanical wear.

Why it happened: PWM (Pulse Width Modulation) signals control the servo. If those signals are noisy or unstable - due to power fluctuations or low-quality wiring - the servo gets confused and "jitters."

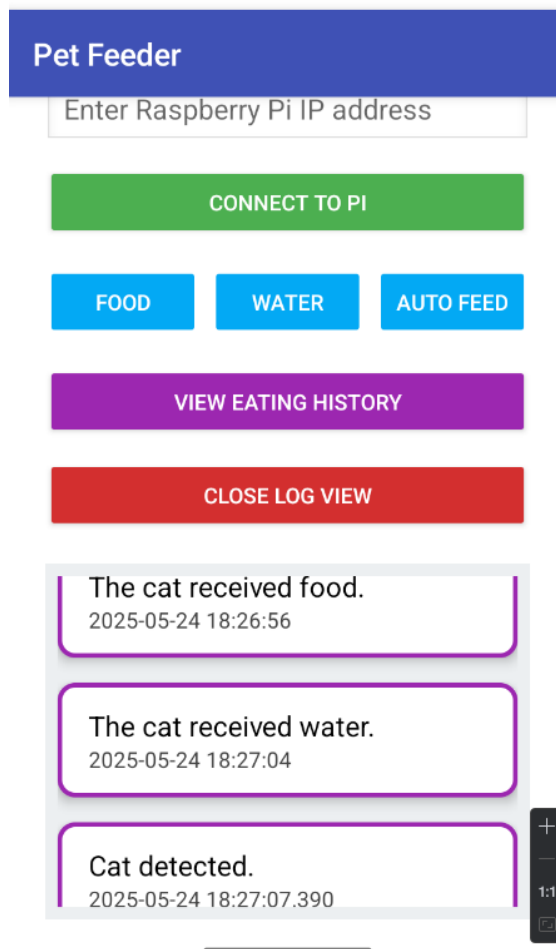
The solution for our problem: PWM signal smoothing via hardware capacitors (electronic components that store and smooth out energy). We figured out that this will help filter out voltage spikes, leading to a cleaner, more stable PWM signal.

14. Future Enhancements

This section outlines planned improvements to evolve the system from a reactive feeder into a smart, predictive, and resilient ecosystem for multi-pet homes.

- *Predictive Analytics:* Integrate Azure Machine Learning models to analyze feeding pattern insights. Use this data to forecast hunger cycles, detect anomalies (e.g., skipped meals), and suggest optimized feeding schedules. Over time, the system can become adaptive to each pet's habits. The system learns - so feeding isn't just on time, it's on *need*.
- *Battery Backup:* UPS (Uninterruptible Power Supply) for power outages. This is especially important for remote users.
- *Voice Alerts:* App notifications for low water or food.

Current Application Screenshot:



15. Conclusion

The Automated Pet Feeder represents a successful integration of IoT hardware, cloud services, and a mobile application into a cohesive system that intelligently manages pet feeding. By combining edge computing capabilities on the Raspberry Pi with real-time data synchronization to Microsoft Azure, the system ensures timely, accurate feeding based on actual consumption data. Features like load cell weight sensing, motion-triggered image recognition, and cat-specific identification via YoloV5 add precision and personalization, distinguishing this feeder from basic timed dispensers.

Additionally, the Android mobile app provides users with an interface for remote monitoring and control, including the ability to view detailed feeding logs and trigger feed events. This enhances the overall usability and reliability of the system, making it practical for everyday pet owners.

Security and reliability have also been addressed through HTTPS communication, IP whitelisting, and Wi-Fi self-recovery mechanisms, ensuring that the system remains functional and protected in typical home environments.

In essence, we think that this project demonstrates how thoughtful design, practical engineering, and cloud-connected intelligence can solve a real-world problem in a way that is scalable, secure, and user-friendly.

